

Détection de comportements suspects par IA (YOLOv8)

Entraînement d'un modèle YOLOv8 par transfer learning pour repérer un vol à l'étalage sur un flux caméra, servi par une API Flask avec interface web et alerte SMS.

CADRE

Projet BTS SIO

MODÈLE

YOLOv8n (Ultralytics)

CLASSES

normal · shoplifting

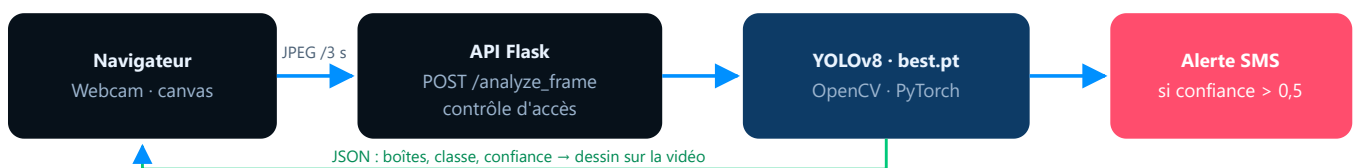
MAP@50

0,894

01 Objectif

Automatiser une tâche d'analyse visuelle : détecter, image par image, un comportement de vol à l'étalage (*shoplifting*) sur le flux d'une caméra de magasin, puis prévenir immédiatement un responsable. Le projet couvre toute la chaîne : jeu de données, entraînement, évaluation, API d'inférence et interface web.

02 Architecture



03 Jeu de données

Élément	Détail
Source	Roboflow Universe, projet « shoplifting » (smartgoggle), version 1, licence CC BY 4.0
Classes	normal, shoplifting
Découpage	Ensembles train / valid / test décrits dans data.yaml
Format	Annotations YOLO (boîtes englobantes normalisées)

04 Entraînement (transfer learning)

Le modèle part des poids pré-entraînés `yolov8n.pt` (COCO), qui savent déjà repérer des formes et des personnes : le réseau est seulement réajusté sur les deux classes du projet, ce qui réduit fortement le temps d'apprentissage et la quantité d'images nécessaire.

Hyperparamètre	Valeur	Hyperparamètre	Valeur
Modèle de base	yolov8n.pt	Époques	100
Taille d'image	640 px	Batch	16
Optimiseur	auto	Learning rate initial	0,01
Mosaïque	1,0	Retournement horizontal	0,5

Translation / échelle

0,1 / 0,5

Variation HSV (t, s, v)

0,015 / 0,7 / 0,4

```
from ultralytics import YOLO
```

```
model = YOLO("yolov8n.pt")
```

```
# poids pré-entraînés COCO
```

```
model.train(data="dataset/data.yaml", epochs=100, imgsz=640, batch=16,
```

```
name="shoplifting_model")
```

```
# best.pt et last.pt dans runs/detect/
```

05 Résultats sur le jeu de validation

0,865

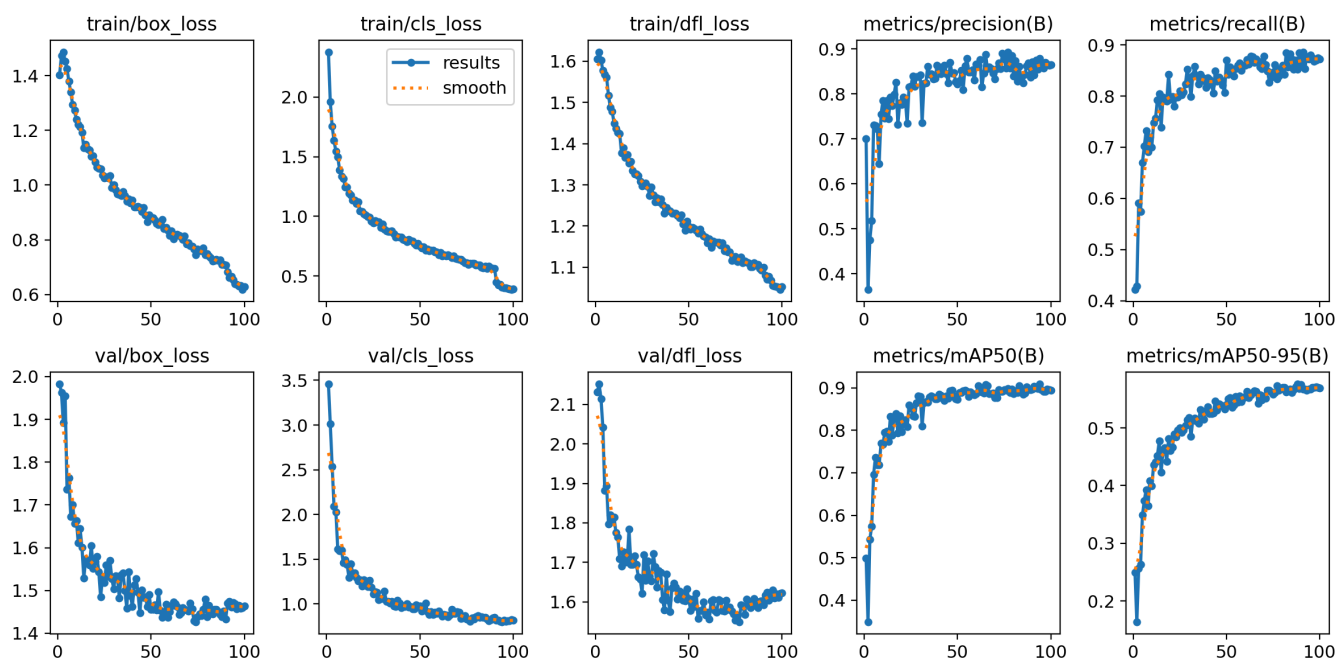
précision

0,872

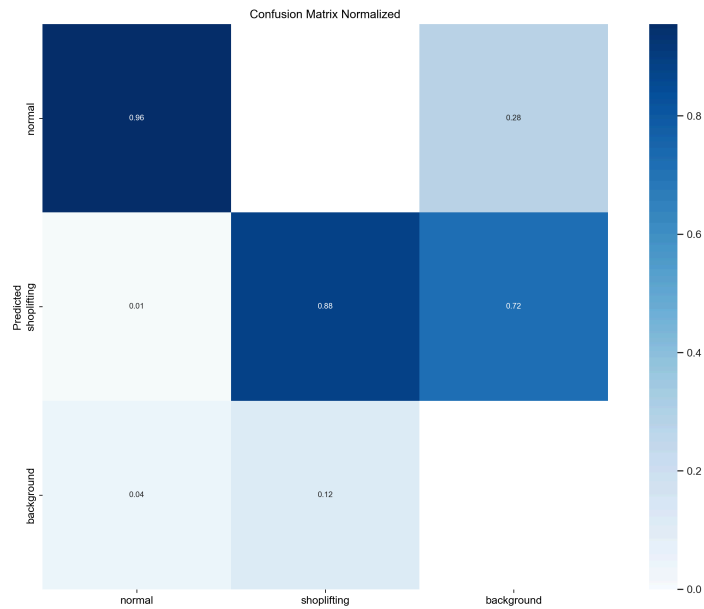
rappel

0,894mAP@50 (meilleur : 0,909 à
l'époque 94)**0,569**

mAP@50-95



Courbes d'apprentissage : pertes (box, cls, dfl) et métriques sur 100 époques.



Matrice de confusion normalisée sur le jeu de validation.



Exemples de prédictions du modèle sur des images de validation.

06 Déploiement : API Flask et interface web

- **API** : la route `POST /analyze_frame` reçoit une image, la decode avec OpenCV, lance l'inférence et renvoie en JSON les coordonnées, la classe et la confiance de chaque détection.
- **Interface** : la page web active la webcam, envoie une image toutes les 3 secondes et dessine les boîtes et les étiquettes sur un canvas superposé à la vidéo.
- **Alerte** : si la classe `shoplifting` dépasse 50 % de confiance, un SMS est envoyé au responsable via l'API Twilio.

```
@app.route("/analyze_frame", methods=["POST"])
def analyze_frame():
    if request.headers.get("Secret-Code") != SECRET_CODE:
        return jsonify({"error": "Unauthorized"}), 401
    frame = cv2.imdecode(np.frombuffer(request.files["frame"].read(), np.uint8), cv2.IMREAD_COLOR)
    results = model(frame)
    # ... construction de la liste des détections et alerte éventuelle
```

07 Sécurité et améliorations apportées

SECRETS RETIRÉS DU CODE

La version publiée lit les identifiants Twilio, les numéros de téléphone, le code d'accès de l'API et le chemin du modèle dans des variables d'environnement (`TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `API_SECRET_CODE` ...). Aucun secret n'est stocké dans le dépôt.

- Mode debug de Flask désactivé par défaut.
- Pistes d'évolution : authentification par jeton côté serveur plutôt qu'un code partagé dans le JavaScript, HTTPS, anti-rebond sur les alertes (une seule alerte par minute), historique des détections avec horodatage.
- Limites : jeu de données public et restreint, deux classes seulement ; un déploiement réel exigerait une analyse d'impact RGPD (vidéosurveillance) et une validation humaine de chaque alerte.

08 Technologies

Python

Ultralytics YOLOv8

PyTorch

OpenCV

NumPy

Flask

JavaScript (getUserMedia, Canvas)

Twilio

Roboflow